YASH BHURKE, 04

AIM: Cryptography in Blockchain, Merkle root Tree Hash

THEORY : A cryptographic hash function takes an input of arbitrary length and produces a fixed-length output called a hash or digest. In blockchain, SHA-256 (Secure Hash Algorithm 256-bit) is widely used. Key properties include determinism (same input always gives same output), avalanche effect (a tiny change in input drastically changes the output), pre-image resistance (you cannot reverse-engineer the input from the hash), and collision resistance (no two different inputs produce the same hash). These properties make hashes ideal for ensuring data integrity across a distributed network.

Nonce and Proof-of-Workmn

A nonce (Number Used Once) is an integer that miners repeatedly increment and combine with block data to produce a hash that satisfies the network's difficulty target — typically a hash beginning with a certain number of leading zeros. This target hash is called the Golden Nonce. The process of searching for this nonce is computationally expensive by design, requiring significant trial-and-error. This is the foundation of the Proof-of-Work consensus mechanism used in Bitcoin, ensuring that adding a new block to the chain demands real computational effort, making tampering prohibitively costly.

Merkle Tree

A Merkle Tree is a binary tree of hashes. Each leaf node holds the hash of an individual transaction. Each non-leaf (parent) node holds the hash of the concatenation of its two child hashes. This process repeats level by level until a single hash remains at the top — the Merkle Root. If the number of nodes at any level is odd, the last node is duplicated before pairing. The Merkle Root is stored in the block header and acts as a compact cryptographic summary of all transactions in that block.

How It Works

Given transactions T1, T2, T3, T4, T5:

1. Each transaction string is individually hashed → leaf hashes H1 through H5.
2. H5 is duplicated to make the count even → H1-H2 combined, H3-H4 combined, H5-H5 combined.
3. The resulting three hashes are paired again (duplicating the odd one out) and hashed.

4. This continues until one final hash remains — the Merkle Root.

Benefits of Merkle Trees

- Efficient verification: any single transaction can be verified without downloading the full block using a Merkle Proof.
- Tamper detection: altering any transaction changes its leaf hash, which propagates up and changes the root, making tampering immediately detectable.
- Bandwidth savings: light clients (SPV nodes) only need the root hash and a small proof path, not the entire blockchain.

Use Cases

- Bitcoin and Ethereum use Merkle Trees to summarize block transactions.
- Git uses a similar Merkle DAG structure to track file changes.
- Certificate transparency logs use Merkle Trees to prove certificate inclusion.
- Distributed file systems like IPFS use content-addressed Merkle DAGs for data integrity.

OUTPUTS :

Program 1:

```
import hashlib                      # fixed

def create_hash(string):

   hash_object = hashlib.sha256()

   hash_object.update(string.encode('utf-8'))

   hash_string = hash_object.hexdigest()    # fixed

   return hash_string


input_string = input("Enter a string: ")
```

```
hash_result = create_hash(input_string)

print("Hash:", hash_result)          # fixed

Enter a string: Yash

Hash: 48a534e64e3b91484555483f2b6bea7069261bf6b2e363b69b9bdcf2d15684fa
```

Program 2:

```
import hashlib


# Get user input

input_string = input("Enter a string: ")

nonce = input("Enter the nonce: ")


# Concatenate the string and nonce

hash_string = input_string + nonce  # Error


# Calculate the hash using SHA-256

hash_object = hashlib.sha256(hash_string.encode("utf-8"))


hash_code = hash_object.hexdigest()

# Print the hash code

print("Hash Code:", hash_code)  # Error


Enter a string: yash
```

```
Enter the nonce: 9

Hash Code: d09e81445859b10bce49fc4b1a5ca8afaf59b102e0dffd27576e386e578ba184
```

Program 3:

```
import hashlib


def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = "0" * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)  # Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode("utf-8"))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):  # Error
            print("Hash:", hash_code)
            return nonce

        nonce += 1
```

```
# Get user input

input_string = "A"

num_zeros = 1


# Find the expected nonce

expected_nonce = find_nonce(input_string, num_zeros)


# Print the expected nonce

print("Input String:", input_string)

print("Leading Zeros:", num_zeros)

print("Expected Nonce:", expected_nonce)  # Error


Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fbb

Input String: A

Leading Zeros: 1

Expected Nonce: 9
```

Program 4:

```
import hashlib
```

```
def build_merkle_tree(transactions):  # fixed

    if len(transactions) == 0:

        return None


    if len(transactions) == 1:

        return transactions[0]


    while len(transactions) > 1:

        if len(transactions) % 2 != 0:

            transactions.append(transactions[-1])


        new_transactions = []

        for i in range(0, len(transactions), 2):

            combined = transactions[i] + transactions[i + 1]

            hash_combined = hashlib.sha256(combined.encode("utf-8")).hexdigest()

            new_transactions.append(hash_combined)  # fixed


        transactions = new_transactions


    return transactions[0]



transactions = [
```

```
    "Transaction 1",
    "Transaction 2",
    "Transaction 3",
    "Transaction 4",
    "Transaction 5",
]

merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)  # fixed
```

```
Merkle Root: a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077
```